Earned Autonomy

Conditioning Agent Trust on Delivered Outcomes, Not Task Accuracy

Andrew Zigler*

July 2026

Executive summary

Enterprises have adopted autonomous software agents faster than they have learned to tell whether the agents are delivering. The dominant answer to that trust problem, published under names like earned, graduated, and progressive autonomy, conditions an agent's authority on how accurately it performs its tasks. This paper argues that task accuracy is the wrong adaptive signal. Autonomy conditioned on task accuracy re-creates at the trust layer the same clean-record trap the audit layer already has: a fully authorized, correctly typed, cleanly traced call can open a defective change while every record reads clean, and an accuracy-graded ladder promotes a workflow at the moment its output draws the least scrutiny. Delivered outcome, whether an agent's change held in production, is the adaptive conditioning variable these autonomy ladders leave out. It complements the preventive gates that screen every change regardless of a workflow's record; it does not replace them. The Model Context Protocol revision finalizing on 2026-07-28 is the first open protocol standard to make per-action agent-tool authorization a first-class, session-independent property, so a workflow's earned tier can be enforced per action while its outcomes are evaluated on a cadence. The paper walks what the release makes legible and enforceable, defines a *delivery evidence layer* that joins governed actions to their delivered outcomes, and describes earned autonomy: a rolling trust cadence, modeled on the standard's own governance by measurement, that grants and revokes agent authority on evidence.

1 The question that predates the spec

Organizations have poured budget, headcount, and roadmap into autonomous software work, and most of them still cannot say whether it paid off. MIT Media Lab's Project NANDA, surveying roughly three hundred enterprise generative-AI initiatives, found that about 95% showed no measurable P&L return [2]. That figure has many causes. One of them is plain: a return you cannot attribute is a return you cannot claim. The 95% is, in part, an attribution failure.

Adoption is not the constraint. Stanford's AI Index put enterprise AI adoption at 78% in its 2025 edition and 88% a year later [15, 16]. The tools are in the building. What remains unanswered is whether the autonomous work they produce is delivering, and for whom.

Scale is what makes the gap urgent. A single developer reviewing a single assistant's suggestions is a manageable trust problem. A fleet of agents opening changes across dozens of repositories, at a rate no human reads line by line, is not. Trust at that scale has to be granted by policy rather than by inspection, and policy needs a signal to run on. The governance question here is older than any protocol. Before a wire format made agent activity legible, leaders already needed to know which autonomous work had earned more scope and which had not. The Model Context Protocol revision finalizing on 2026-07-28 does not create that question. It changes what an organization can answer it with.

*The author is employed by LinearB, which produced the 2026 benchmark cited herein. LinearB is not affiliated with the standards or projects this paper discusses.

The question also already has a named answer in the field, and that answer is worth taking seriously before improving on it. Earned autonomy, graduated autonomy, and progressive autonomy all describe the same ladder: an agent starts under human review, and as it performs, it earns the right to act with less oversight. Matthias Röder's earned-autonomy gradient formalizes the rungs [14]; a growing set of platform vendors ship progressive-autonomy controls [1]; the Digital Apprentice line of work models the same apprenticeship arc [18]; and McKinsey frames agent trust as decision rights granted and revoked over time [6].

Every approach we surveyed conditions autonomy on the same class of signal: how well the agent performed its task. Extraction accuracy, override rate, tool-call correctness. That choice of variable is the one thing this paper revisits, because it is the same variable that lets a clean record hide a bad outcome. This paper keeps the name earned autonomy and changes only the variable it conditions on: delivered outcome, not task accuracy. The rest of the argument shows why the 2026-07-28 spec is the first open protocol standard on which that condition can be enforced per action.

2 What the spec makes legible and enforceable

After the 2026-07-28 revision, every agent action is at once a well-formed record and a policy decision point. That, not the size of the release, is what makes it matter for a governance argument. Five properties produce that result. None is invented here; each is a mechanism the release already ships, and the point is what they add up to.

Identity-bound. Every request now carries a client label in its `_meta` block rather than establishing it once per session: the `io.modelcontextprotocol/clientInfo` field travels with the call itself (the Specification Enhancement Proposal SEP-2575) [9]. That label is self-declared by the client, not authenticated. The authenticated principal comes from the authorization side: the release hardens OAuth so a client must validate a present issuer (`iss`) parameter against the recorded authorization server before it redeems a code, closing a class of mix-up attacks (SEP-2468) [9]. An action arrives attributable to an OAuth-authenticated principal, with `clientInfo` adding a client-asserted implementation label that is useful for attribution only when it is bound to an issued credential.

Typed. Tool inputs and outputs are lifted to full JSON Schema 2020-12 (SEP-2106), so the shape of every call and its result is declared and checkable rather than conventional [9]. A record whose payloads validate against a schema is a record a downstream system can parse without guessing at its structure.

Traceable. The release documents W3C Trace Context conventions (`traceparent`, `tracestate`, and `baggage` in `_meta`, per SEP-414), so a single action correlates across SDKs, gateways, and any OpenTelemetry backend [9]. The deprecation of the old Logging primitive points the same way: diagnostic logging moves to `stderr` and OpenTelemetry (SEP-2577) [8]. Observability belongs to the tracing plane, not to a bespoke protocol message.

Enforceable. Because the release removes protocol sessions (SEP-2567), any request can land on any server replica, and authorization is therefore evaluated on every call rather than inherited from a session established once [13]. The release also requires routing headers, `Mcp-Method` and `Mcp-Name`, on Streamable-HTTP POSTs, and requires servers to reject a request whose headers and

body disagree (SEP-2243) [9]. A gateway can now make an allow, deny, or downgrade decision on the operation without parsing the body. The per-call boundary is a real enforcement point.

Fleet-scalable. List and read results now carry required cache metadata: a `ttlMs` freshness hint and a `cacheScope` of `public` or `private` (SEP-2549) [9]. With sessions gone, these results are stable enough to cache, which makes the served context and tool listings cheap to reuse at fleet scale. Authorization stays per-call; caching lowers the data-fetch cost, not the cost of the decision made on every request.

Set these five properties side by side and the shape is clear. An organization running agents on the 2026-07-28 spec gets, from the protocol alone, a stream of actions that are attributable, typed, traceable, individually enforceable, and cheap to observe at fleet scale. This is a genuine advance in what a protocol makes governable, and the temptation is to stop here and call the governance problem solved. It is not solved. A well-formed record has a limit, and that limit is the subject of the next section.

3 What the record cannot tell you

A record can be complete and still be silent about the thing that matters. Consider an agent action that is fully authorized, correctly typed, cleanly traced, and schema-valid, a record with nothing wrong in it, that opens a pull request introducing a SQL injection. Nothing in the protocol layer is equipped to notice, because nothing in the protocol layer looks at what the change does once it is merged. Veracode's 2025 analysis found that 45% of AI-generated code samples introduced an OWASP Top-10 vulnerability, across more than a hundred models, eighty tasks, and four languages, with the rate roughly flat regardless of model size or recency [17]. The defective change and the correct change produce identical protocol records.

This is the clean-record trap, and the audit layer has always had it. A log tells you an action happened and was permitted; it never tells you the action was any good. Conditioning autonomy on task accuracy re-creates that trap one level up, because task accuracy measures the agent's behavior at the call and sees nothing of what the call delivers. A workflow that scores 99% on tool-call correctness is graded on whether its calls are well-formed, not on what they deliver: the defective call *is* a correct call. Worse, an autonomy ladder built on task accuracy promotes a workflow precisely when its accuracy scores are high, which is the moment its output is drawing the least human scrutiny. The metric rises as the examination falls.

What lies past the call divides into two failure classes; Section 5 returns to the division of labor between them. Latent defects, the security, privacy, and license problems the SQL injection stands for, ride clean through every production signal and are caught only by reading the change itself; they belong to the preventive gates. Delivery failures, the reverts, rework, and changes that sit unmerged, are caught only by watching what shipped, and that is this paper's subject. LinearB's 2026 benchmark report, drawn from 2.7 million pull requests across 253 organizations, supplies the field evidence: the output that matters is delivered work, not generated code, and agent-opened pull requests merge at markedly lower rates than human-opened ones, because a change no human owns tends to sit unmerged [7]. The variable that counts lives one step past the tool boundary, at the delivered change, and no protocol call runs there.

4 The delivery evidence layer

Start with a limit that most engineering organizations already know they have. The delivery-metrics programs leaders run today, chief among them DORA's four keys [4], measure delivery at the team level. Designed before agents wrote code, they tell you what is happening, not what drove it: a

change-failure rate can move with no way to attribute the move to AI tooling, process change, or a quality trade-off. Call this the attribution gap: it and the 95% with no measurable P&L return are the same problem seen from two ends, an outcome that cannot be attributed to the work that produced it.

Closing that gap requires a layer the protocol does not provide, and naming it correctly matters, because an adjacent discipline already claimed a similar-sounding term for the opposite job. Context engineering builds a *context layer* whose purpose is to feed better inputs to agents so they act well [5]. The layer this paper describes runs in the other direction. Call it the *delivery evidence layer*: its purpose is to feed outcome evidence to the authorizer (the human or process that makes the trust decision, later the cadence) so the organization can decide what an agent is allowed to do next. Context feeds the actor. Evidence feeds the authorizer. It is one store, and it has four capabilities.

Attribute. Attribute joins the spec's per-call action records to version-control and CI/CD events, so a delivered change carries a legible answer to which autonomous work produced it and through what path. The join is the implementer's to build. The spec emits attributable, traceable action records (SEP-414, SEP-2575) and the pipeline emits commits, reviews, and deploys, but nothing in the protocol stitches the two together. That stitch is the fix for the attribution gap, and it is the foundational capability the other three depend on.

Evaluate. Evaluate attaches outcome signals to each agent-authored change: change-failure rate, revert, rework, override-before-merge, whether the change held in production, and cost-to-outcome. These are delivery signals, not activity counts. They answer a different question than how many actions the agent took. They answer what the actions delivered, and what the delivery cost. Evaluate is where the conditioning variable of the whole argument, delivered outcome, gets computed.

Serve. Serve hands that evidence back as deterministic, typed, freshness-contracted context, delivered over the spec's own substrate. Here the release pays a second dividend: full JSON Schema typing (SEP-2106) makes the served context checkable, required cache scopes and TTLs (SEP-2549) make its freshness a contract rather than a hope, and the per-call boundary (SEP-2567) is where it is delivered and re-authorized. The same protocol that emitted the events carries the answers back, and Serve reaches two consumers over those rails: the review cadence (defined in the next section) reads the evidence as the authorizer's input, and the agents read a distilled view of the same evidence as operating context, the current baselines and constraints they should act within.

Steer. Steer feeds evaluations into boundary policy: the outcomes a workflow has delivered set the autonomy it is granted on its next call. Steer is the capability that turns the join into a loop, and it is where the protocol earns its place in this argument, because the per-call enforcement point is the actuator the loop writes to. It is also the capability most easily overstated, so the next section defines it carefully and states its limits first.

Attribute and Evaluate are the write path; Serve and Steer are the read path. None of the four is speculative: a join over data an organization already has, metrics that teams already track, a typed read over a governed transport, and policy at a boundary the spec now provides. What makes them a layer rather than four disconnected scripts is that they share one store of delivered-outcome evidence, keyed to the unit an organization wants to govern.

5 Earned autonomy

The 2026-07-28 release ships into a conformance-gated, SDK-tiered ecosystem: a conformance suite live since 2026-01-23, published SDK tiers since 2026-02-23, and, as of SEP-2484 (Final 2026-06-04), a standing rule that future Standards-Track SEPs cannot reach Final without a matching conformance scenario [11, 12]. The SDK tiers are the part worth studying. A Tier 1 SDK must pass 100% of the conformance suite; if any test fails continuously for four weeks, it is relegated to Tier 2; and features move through a lifecycle with a minimum twelve-month deprecation window measured from the revision release [10, 11]. Standing is earned by demonstrated conformance and lost publicly when the demonstration lapses.

That is a working model of earned standing, and humans run it: no automated controller relegates an SDK. Maintainers publish tiers against measured conformance, on defined clocks, with a defined relegation rule, and the governance works. This is the precise shape to port. Earned autonomy is a rolling trust review in which workflow configurations (the durable bundle of model, prompt, tools, and policy that produces the work) hold published autonomy tiers, delivery-evidence-layer signals are the input, promotion and relegation follow defined triggers, and the resulting grant is enforced per call at the boundary the spec provides. Steer is that cadence: a review rhythm with a clock and a rule, of the same kind the standard already runs on itself. The closed-loop controller that reads a metric and rewrites policy is the forward agenda, not the claim here.

Every component of this cadence is mechanically expressible today: identity and per-call enforcement from the spec, attribution and outcome evaluation from the delivery evidence layer, published tiers and clocks from process. The only genuinely new input is the swap at the center: replace static scope grants, issued once and rarely revisited, with grants conditioned on delivered-outcome evidence.

Stated plainly: the grant a workflow has earned is enforced per action; its delivered outcome is evaluated on a cadence. That coupling, per-action enforcement of a cadence-evaluated grant, is what no accuracy-graded ladder provides. Delivered outcome is the only *adaptive* conditioning variable that closes the action-to-outcome loop.

Earned autonomy is one of two layers, and it is the adaptive one. Beneath it sits a preventive layer that screens every change before merge regardless of which workflow authored it or what record that workflow holds: pre-merge security scanning, blast-radius and impact gating, and agentic code review. A cadence tuned to delivered outcomes is blind to the latent-defect classes (security, privacy, license) that ride clean through every production signal until they are exploited. The SQL injection of the earlier section holds in production, passes each outcome check, and would earn a workflow more scope on outcome evidence alone. Only a gate that reads the change itself catches it. What the cadence catches is the complementary class: the reverts, rework, and unmerged changes that read clean at the tool boundary and fail only in delivery. Earned autonomy governs how much a workflow is trusted to act without human review over time; it presupposes, and does not replace, the automated screening of every change.

The failure modes are dials, not objections. Because Steer is a cadence, its known failure modes are design parameters that set the cadence's dials. *Attribution lag* (outcomes arrive days or weeks after the change) sets the length of the trailing window the review scores on; instantaneous scoring is the mistake the window length corrects. *Unstable identity* (agent instances are ephemeral and interchangeable) sets the unit of trust: the durable thing to govern is a workflow configuration, not an instance, because the configuration is what produced the work and what will produce it again. *Goodhart's law* sets the gating signal: gate on held-in-production outcomes and never on activity counts, because the moment the metric is a throughput count it is gamed. *Small-n* (too few

observations to trust) sets the cold-start default: absent evidence, a workflow starts in a sandbox tier, since autonomy is earned from evidence and the safe default without it is low.

Two clarifications keep the framework honest. First, the per-call boundary is an enforcement point, not a scope engine. The spec gives the point at which authorization is checked on every call; it does not decide what a given principal may do. The scope semantics, the policy itself, remain the implementer's design. Authentication is not authorization scope. Second, per-call identity attributes to the authenticated principal and the named client implementation, carried in `clientInfo`, and not to an individual agent or subagent. Agent-level attribution is a convention an organization builds on top, and it is exactly why the unit of trust is a workflow configuration: that is the granularity the evidence layer can attribute to, not a per-instance identity the protocol never reported.

This shape has precedent outside software delivery. The CNCF moves a project from Sandbox to Incubating to Graduated on demonstrated adoption, security review, and governance maturity, ratified by humans rather than by a metric gate [3]. That is the posture earned autonomy takes toward a workflow: evidence-based maturation with human ratification.

There is also empirical support for building the feedback capability the framework depends on. DORA's 2025 research found that AI acts as an amplifier: it has a positive relationship with delivery throughput and a negative one with stability, and which effect dominates is conditional on a team's existing capabilities [4]. Among those capabilities is the delivery-feedback capability this paper argues an organization must build. The amplifier result corroborates that delivery stability is the thing at risk. It motivates caring about outcomes; it does not, by itself, pick the conditioning variable.

The cadence is configuration, not research. The trust decision, reading the delivered-outcome evidence and setting the tier, is the organization's to make; the gateway's only job is to enforce the resulting scope on every call. The gateway never consults evidence at call time: the cadence compiles evidence into an identity assignment, and the gateway enforces that identity. A workflow holds the identity its current tier has earned, and the gateway gates each tool call against that identity. Because a tier is bound to a credential, relegation is not complete until the prior tier's credential is revoked, not merely superseded: a workflow that keeps a live higher-tier key keeps the scope it was supposed to lose. Issuing, rotating, and revoking those credentials is the organization's responsibility, not the protocol's.

6 Recommendations

The move a leader can make on Monday costs no procurement. It is to stand up the cadence as a process before it is a product. Concretely: publish autonomy tiers for the workflow configurations already running agents, from a sandbox tier up through a trusted tier; define the trailing window and the outcome signals each tier is scored on; set the relegation triggers and the clock, borrowing the standard's own four-week shape as a starting point; and default every new or sparsely-observed workflow to the sandbox. This is a review rhythm a platform or developer-experience team can run in a spreadsheet on the first day and encode in gateway policy as it matures. None of it requires buying anything.

The cadence adjusts trust over time; it presupposes a gate that screens every change regardless of the workflow's record, and that gate is the cheapest control to stand up. Turn on pre-merge security scanning, blast-radius and impact gating, and agentic code review on every pull request, agent-authored or not. This is the preventive layer earned autonomy sits on; it is not an alternative to the cadence.

The capability to build alongside it is the delivery evidence layer, and it belongs in the same

```
# Per-call authorization at an MCP gateway. The cadence maps each
# workflow to a tier identity at credential issuance; the gateway
# enforces that identity's scope on every tool call. Relegation =
# the cadence reissues the workflow a lower-tier identity (no edit
# to this policy), and its next call is gated anew.
authorization:
  - allow: identity == "tier-trusted"       # full access
  - allow: identity == "tier-sandbox"       # read-only
        && tool in ["read_file", "list_directory", "search_files"]
```

Figure 1: **Actuator validated on agentgateway**, an open-source MCP gateway (an AAIF project). A per-call authorization rule gates tool scope on a workflow's tier-earned identity; the identical `write_file` call was allowed for the trusted identity and denied, per call, for the sandbox identity, because `write_file` is absent from that tier's allowlist, while the read call was allowed for both. This validates the actuator, the per-call identity-scoped authorization point; the sensor (the delivery evidence layer) and the policy (the cadence) are described in this paper, not demonstrated here. The pattern needs only per-call, identity-scoped policy and runs on any gateway with that capability. It is the shape teams already use to gate a deploy, pointed at delivered-outcome evidence.

budget cycle as the MCP boundary work, not after it. The boundary makes actions enforceable; the layer makes outcomes legible; the cadence needs both. Sequencing the enforcement work first and the evidence work later produces the exact state this paper opened on: a fleet whose actions are perfectly recorded and whose value is unattributable.

When evaluating any system, internal or purchased, that claims to govern agent autonomy, five questions separate an outcome-conditioned tool from an activity-graded one:

- **Attribute at the change level.** Does it join governed actions to the commit, review, and deploy they produced, or does it stop at the action record?
- **Close the action-to-outcome loop.** Can a granted scope be traced to the outcome it caused, so authority and result meet in one place?
- **Segment authorship.** Does it tell agent-authored change apart from human-authored change, so the attribution gap is closed rather than aggregated over?
- **Resist adoption proxies.** Does it condition trust on held-in-production outcomes and refuse activity and adoption counts, such as action volume or acceptance rate, as stand-ins for delivery?
- **Serve context over governed rails.** Does it return evidence to the agents over the same typed, cache-contracted, per-call-authorized transport the actions ran on?

The 2026-07-28 spec settles the question of whether an organization can see and control what its agents do, action by action. It leaves open the question this paper is about: whether the organization can tell what those actions delivered, and condition trust on the answer. The substrate for that second question now exists. The layer above it, and the cadence above that, are the work in front of every team running agents at scale.

References

[1] Agent platform vendors (industry). Progressive autonomy controls. Representative industry usage; see e.g. https://www.mindstudio.ai/ and https://mightybot.ai/, 2025. Autonomy tiers gated on agent task performance.

[2] Aditya Challapally, Chris Pease, Ramesh Raskar, and Pradyumna Chari. The GenAI divide: State of AI in business 2025. https://www.media.mit.edu/groups/nanda/overview/, 2025. MIT Media Lab, Project NANDA; July 2025.

[3] Cloud Native Computing Foundation. Project lifecycle: Sandbox, incubating, graduated. https://contribute.cncf.io/projects/lifecycle/, 2025.

[4] DORA (Google Cloud). State of AI-assisted software development 2025. https://dora.dev/research/2025/, 2025.

[5] IBM. The structured context layer for enterprise AI. https://www.ibm.com/think, 2025.

[6] Rich Isenberg and Lucia Rahilly. Trust in the age of agents. https://www.mckinsey.com/capabilities/risk-and-resilience/our-insights/trust-in-the-age-of-agents, 2026. McKinsey & Company, McKinsey Podcast, 5 March 2026.

[7] LinearB. The engineering productivity gap: The 2026 AI benchmarks. LinearB; report URL pending publication, 2026. 2.7M PRs across 253 organizations, Feb–May 2026.

[8] Model Context Protocol Maintainers. Deprecated features registry (2026-07-28 revision). https://modelcontextprotocol.io/specification/draft/deprecated, 2026. SEP-2577: Roots, Sampling, Logging deprecated.

[9] Model Context Protocol Maintainers. Specification draft changelog (2026-07-28 revision). https://modelcontextprotocol.io/specification/draft/changelog, 2026. SEP-2575, SEP-2468, SEP-2106, SEP-414, SEP-2243, SEP-2549.

[10] Model Context Protocol Maintainers. Feature lifecycle and deprecation policy (SEP-2596). https://modelcontextprotocol.io/community/feature-lifecycle, 2026.

[11] Model Context Protocol Maintainers. SDK tiers and conformance. https://modelcontextprotocol.io/community/sdk-tiers, 2026.

[12] Model Context Protocol Maintainers. SEP-2484: Conformance scenario required for standards-track Final. https://github.com/modelcontextprotocol/modelcontextprotocol/pull/2484, 2026. Final 2026-06-04.

[13] Model Context Protocol Maintainers. SEP-2567: Sessionless MCP via explicit state handles. https://modelcontextprotocol.io/seps/2567-sessionless-mcp, 2026.

[14] Matthias Röder. The earned autonomy gradient: When can you trust AI to act alone? https://matthiasroder.com/the-earned-autonomy-gradient-when-can-you-trust-ai-to-act-alone-2/, 2026. Published 2026-02-04.

[15] Stanford HAI. The AI index report 2025. https://hai.stanford.edu/ai-index/2025-ai-index-report, 2025.

[16] Stanford HAI. The AI index report 2026. https://hai.stanford.edu/ai-index/2026-ai-index-report, 2026.

[17] Veracode. 2025 GenAI code security report. https://www.veracode.com/blog/genai-code-security-report, 2025.

[18] Travis Weber and Rohit Taneja. The digital apprentice: A framework for human-directed agentic AI development. https://arxiv.org/abs/2606.04321, 2026.